



**UNIVERSIDAD CENTROAMERICANA
JOSÉ SIMEÓN CAÑAS**

ASIGNATURA:

BASES DE DATOS

PROYECTO:

SISTEMA DE GESTIÓN DE BIBLIOTECA

INTEGRANTES:

LEANDRO JAVIER AQUINO MERINO	00036725
MERLIN ROSMERY CAMPOS MERLOS	00020725
JAQUELINE NICOLE CARDOZA MALDONADO	00252125
CARLOS DANIEL MARTINEZ ROJAS	00032225
MICHAEL ANTONIO VALLE VILLALOBOS	00185223

CATEDRATICO:

ING. JAMES EDWARD HUMBERSTONE MORALES

SECCION: 02

CICLO 01-2026

Indice

1. Descripción del Escenario y Reglas de Negocio	4
1.1 Descripción General	4
1.2 Entidades Clave.....	4
1.3 Reglas de Negocio	4
2. Diagrama Entidad–Relación (Notación Chen)	5
2.1 Diagrama	5
2.2 Relaciones y Cardinalidades	5
3. Esquema Relacional Normalizado (3FN)	6
3.1 Tablas del Modelo	6
Editorial.....	6
Categoria	6
Autor	6
Libro.....	6
Libro_Autor.....	6
Ejemplar.....	6
Tarifa_Multa	6
Socio.....	6
Empleado.....	6
Prestamo.....	7
Multa.....	7
Tarifa_multa	7
4. Diccionario de Datos	8
4.1 Tabla: Editorial.....	8
4.2 Tabla: Categoria	8
4.3 Tabla: Autor	8
4.4 Tabla: Socio	8
4.5 Tabla: Empleado.....	9
4.6 Tabla: Tarifa_multa.....	9
4.7 Tabla: Libro	9
4.8 Tabla: Ejemplar.....	9
4.9 Tabla: Libro_autor.....	10
4.10 Tabla: Prestamo	10
4.11 Tabla: Multa	10
5. Consultas SQL Frecuentes	11
5.1 Top 20 libros más prestados los últimos 3 meses	11

5.2 Socios con multas pendientes	11
5.3 Autores con mayor número de títulos en catálogo	11
5.4 Ejemplares que nunca han sido prestados	11
5.5 Empleado que ha procesado más préstamos en el mes	11
5.6 Socios con más libros prestados	12
5.7 Categoría con más ejemplares	12
6. Funciones, Procedimientos Almacenados y Disparadores	12
6.1 Script de Mantenimiento de Secuencias	12
6.2 Funciones	12
fn_actualizar_limite_prestamo() RETURNS TRIGGER	12
fn_actualiza_estado_prestamo() RETURNS TRIGGER	13
fn_actualiza_estado_multa() RETURNS TRIGGER	13
fn_consulta_categoria(p_nombre_categoria VARCHAR) RETURNS TABLE(isbn_libro VARCHAR, titulo_libro VARCHAR)	13
6.3 Procedimientos Almacenados	14
sp_calcular_multas(p_id_tarifa BIGINT)	14
sp_reserva_libro(p_id_ejemplar BIGINT, p_id_socio BIGINT, p_id_empleado BIGINT)	14
6.4 Disparadores (Triggers)	15
trg_actualizar_limite_prestamo — BEFORE INSERT ON Prestamo	15
trg_actualiza_estado_prestamo — AFTER UPDATE ON Prestamo	15
trg_actualiza_estado_multa — AFTER UPDATE ON Multa	15

1. Descripción del Escenario y Reglas de Negocio

1.1 Descripción General

Una biblioteca pública requiere un sistema para gestionar su catálogo de libros, socios, préstamos, devoluciones, multas y reservas de material bibliográfico. El sistema garantiza la integridad de los datos, automatiza el cálculo de multas por retraso y permite consultas eficientes sobre el estado del inventario y el comportamiento de los socios.

1.2 Entidades Clave

Entidad	Descripción
Libro	Material bibliográfico identificado por ISBN
Autor	Persona que escribe uno o varios libros
Editorial	Empresa que publica libros
Categoría	Clasificación temática de los libros
Ejemplar	Copia física de un libro disponible en la biblioteca
Socio	Persona registrada que puede realizar préstamos
Préstamo	Registro de entrega de un ejemplar a un socio
Multa	Cargo económico por devolución tardía
Tarifa_Multa	Tabla de tarifas de cobro por día de retraso
Empleado	Personal que gestiona los préstamos

1.3 Reglas de Negocio

Las siguientes reglas están codificadas como restricciones (constraints) directamente en el script de creación de la base de datos:

- Un libro puede tener múltiples autores (resuelto mediante la tabla Libro_autor) y varios ejemplares físicos en estantería.
- Cada ejemplar físico solo puede estar en uno de dos estados: “Disponible” o “Reservado” (constraint chk_ejemplar_estado).
- Un préstamo solo puede estar en uno de tres estados: “Prestado”, “Devuelto” o “Vencido” (constraint chk_prestamo_estado).
- La fecha límite de un préstamo nunca puede ser anterior a la fecha en que se realizó (constraint chk_fechas_prestamo).
- La fecha de devolución, si existe, nunca puede ser anterior a la fecha de préstamo (constraint chk_fechas_devolucion).
- Una multa solo se genera cuando existen días de retraso reales y mayores a cero (constraint chk_dias_retraso).
- El monto de toda multa debe ser mayor o igual a cero; no se permiten montos negativos (constraint chk_monto_positivo).
- Cada préstamo puede generar como máximo una multa, garantizado por la restricción UNIQUE sobre id_prestamo en la tabla Multa.

- Una multa solo puede tener estado “Pendiente” o “Pagada” (constraint chk_estado_pago).
- Un socio solo puede tener estado “Activo” o “Inactivo” (constraint chk_socio_estado).
- El año de publicación de un libro debe ser mayor o igual a 1400 y no puede ser posterior al año actual (constraint chk_anio_valido).
- El precio por día de una tarifa de multa no puede ser negativo (constraint chk_tarifa_precio).
- Los nombres de Editorial y Categoría, así como el email de Socio, son valores únicos en el sistema (constraints UNIQUE).

2. Diagrama Entidad–Relación (Notación Chen)

2.1 Diagrama

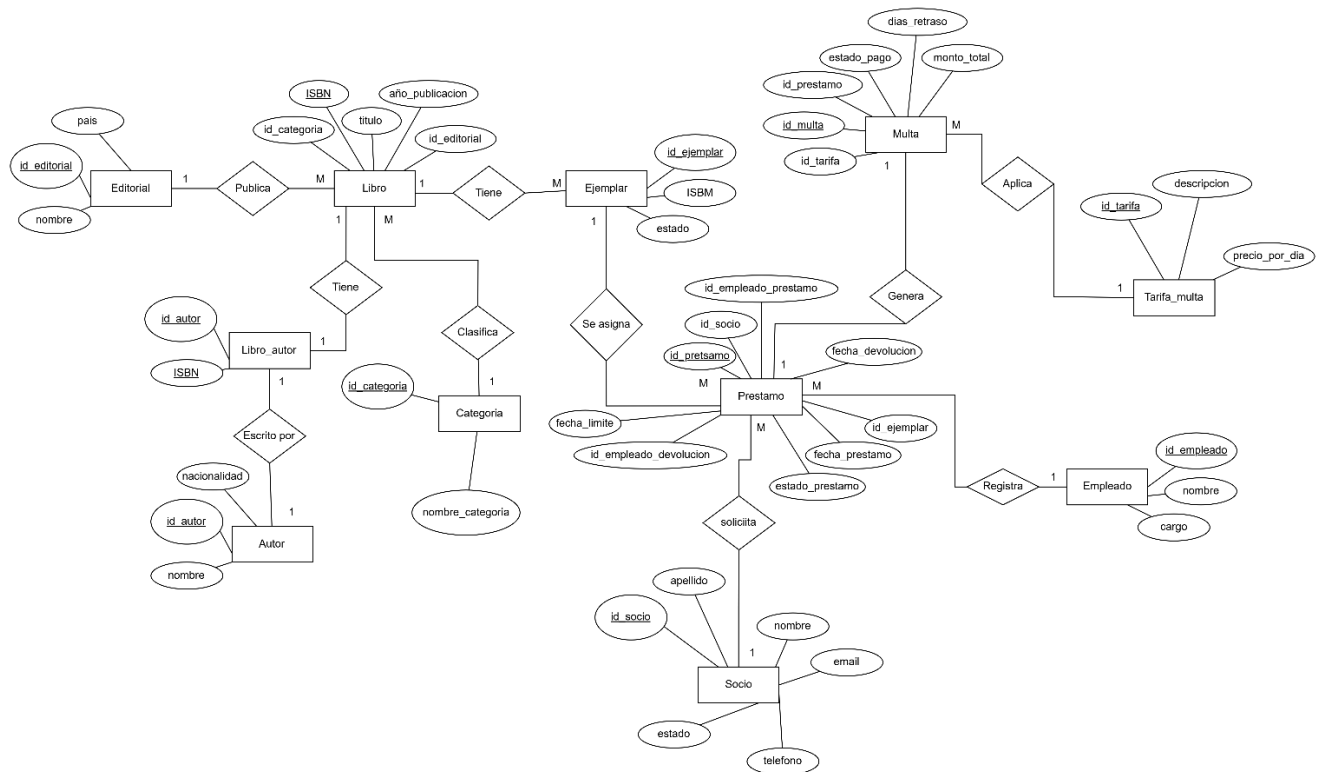


Figura 1. Diagrama ER en notación Chen — Sistema de Gestión de Biblioteca

2.2 Relaciones y Cardinalidades

Relación	Entidades	Cardinalidad	Descripción
Publica	Editorial – Libro	1 : M	Una editorial publica muchos libros
Tiene	Libro – Ejemplar	1 : M	Un libro tiene varios ejemplares físicos
Clasifica	Libro – Categoría	M : 1	Varios libros pertenecen a una categoría
Escrito por	Libro – Autor	M : M	Resuelta con tabla intermedia Libro_Autor
Se asigna	Ejemplar – Préstamo	1 : M	Un ejemplar puede tener varios préstamos a lo largo del tiempo
Solicita	Socio – Préstamo	1 : M	Un socio realiza múltiples préstamos

Registra	Empleado – Préstamo	1 : M	Un empleado entrega y/o recibe múltiples préstamos (FK doble: prestamo y devolución)
Genera	Préstamo – Multa	1 : 0 : 1	Un préstamo genera como máximo una multa (UNIQUE id_prestamo)
Aplica	Multa – Tarifa_Multa	M : 1	Varias multas usan la tarifa vigente
Solicita	Socio – Prestamo	1 : M	Un socio realiza múltiples préstamos

3. Esquema Relacional Normalizado (3FN)

El modelo relacional implementado en PostgreSQL cumple la Tercera Forma Normal (3FN). Cada tabla posee una clave primaria bien definida (simple o compuesta), las claves foráneas garantizan la integridad referencial mediante restricciones explícitas ON DELETE/ON UPDATE, y no existen dependencias transitivas entre atributos no-clave.

3.1 Tablas del Modelo

Editorial

```
id_editorial (PK), nombre [UNIQUE], pais
```

Categoria

```
id_categoria (PK), nombre_categoria [UNIQUE]
```

Autor

```
id_autor (PK), nombre, nacionalidad
```

Libro

```
ISBN (PK), titulo, anio_publicacion, id_editorial (FK), id_categoria (FK)
```

Libro_Autor

```
ISBN (PK, FK), id_autor (PK, FK)– tabla intermedia M:M
```

Ejemplar

```
id_ejemplar (PK), ISBN (FK), estado
```

Tarifa_Multa

```
id_tarifa (PK), descripcion, precio_por_dia
```

Socio

```
id_socio (PK), nombre, apellido, email [UNIQUE], telefono, estado
```

Empleado

```
id_empleado (PK), nombre, cargo
```

Prestamo

```
id_prestamo (PK), id_socio (FK), id_empleado_prestamo (FK), id_empleado_devolucion (FK),  
id_ejemplar (FK), estado_prestamo, fecha_prestamo, fecha_devolucion, fecha_devoulucion
```

Multa

```
id_multa (PK), id_prestamo (FK, UNIQUE), id_tarifa (FK), dias_retraso, monto_total,  
estado_pago
```

Tarifa_multa

```
id_tarifa (PK), descripcion , precio_por_dia
```

3.2 Comportamiento de Integridad Referencial

El script define explícitamente el comportamiento ante eliminación y actualización en cada clave foránea:

- **ON DELETE RESTRICT:** aplicado en Libro→Editorial, Libro→Categoria, Ejemplar→Libro, Prestamo→Ejemplar/Socio/Empleado y Multa→Prestamo/Tarifa_multa. Impide borrar un registro padre si existen hijos dependientes, protegiendo el historial de operaciones.
- **ON DELETE CASCADE:** aplicado únicamente en Libro_autor→Autor y Libro_autor→Libro. Si se elimina un autor o un libro, su relación de autoría se elimina automáticamente sin afectar el registro principal del otro lado.
- **ON UPDATE CASCADE:** aplicado en Libro→Editorial, Libro→Categoria y Multa→Prestamo/Tarifa_multa, propagando cambios de clave automáticamente.

4. Diccionario de Datos

A continuación, se detalla cada columna de las 11 tablas del modelo, según se definen literalmente en el script DDL de PostgreSQL, incluyendo tipos de datos exactos y las restricciones CHECK / UNIQUE / FOREIGN KEY aplicadas.

4.1 Tabla: Editorial

Campo	Tipo	PK	FK / Restricción	Nulo	Descripción
id_editorial	BIGINT IDENTITY	PK		NO	Identificador único de la editorial
Nombre	VARCHAR(150)		UNIQUE	NO	Nombre de la editorial
País	VARCHAR(80)			NO	País de origen de la editorial

4.2 Tabla: Categoría

Campo	Tipo	PK	FK / Restricción	Nulo	Descripción
id_categoria	BIGINT IDENTITY	PK		NO	Identificador único de categoría
nombre_categoria	VARCHAR(80)		UNIQUE	NO	Tipo de categoría del libro ("Terror, Romance, Historia")

4.3 Tabla: Autor

Campo	Tipo	PK	FK / Restricción	Nulo	Descripción
id_autor	BIGINT IDENTITY	PK		NO	Identificador único del autor
nombre	VARCHAR(100)			NO	Nombre completo del autor
nacionalidad	VARCHAR(60)				País de origen del autor

4.4 Tabla: Socio

Campo	Tipo	PK	FK / Restricción	Nulo	Descripción
Id_socio	BIGINT IDENTITY	PK		NO	Identificador único del socio
nombre	VARCHAR(80)			NO	Nombre del socio
apellido	VARCHAR(80)			NO	Apellido del socio
email	VARCHAR(120)		UNIQUE	NO	Correo electrónico del socio
teléfono	VARCHAR(20)			NO	Número de teléfono de contacto
estado	VARCHAR(20)		CHECK in ('Activo','Inactivo')	NO	Estado del socio (Por defecto activo)

4.5 Tabla: Empleado

Campo	Tipo	PK	FK / Restricción	Nulo	Descripción
Id_empleado	BIGINT IDENTITY	PK		NO	Identificador único del empleado
nombre	VARCHAR(100)			NO	Nombre completo del empleado
cargo	VARCHAR(60)			NO	Cargo que ejerce en la biblioteca

4.6 Tabla: Tarifa_multa

Campo	Tipo	PK	FK / Restricción	Nulo	Descripción
id_tarifa	BIGINT IDENTITY	PK		NO	Identificador autogenerado de la tarifa
descripcion	VARCHAR(100)			NO	Descripción del tipo de tarifa
precio_por_dia	NUMERIC(5 ,2)		CHECK >= 0	NO	Costo por cada día de retraso; no admite negativos

4.7 Tabla: Libro

Campo	Tipo	PK	FK / Restricción	Nulo	Descripción
ISBN	VARCHAR(20)	PK		NO	Identificador internacional del libro
titulo	VARCHAR(200)			NO	Título completo del libro
anio_publicacion	INT		CHECK >= 1400 y <= año actual	NO	Año en que fue publicado; valida en rango
id_editorial	BIGINT		FK→Editorial (RESTRICT/CASCADE)	NO	Editorial que publicó el libro
id_categoria	BIGINT		FK→Categoria (RESTRICT/CASCADE)	NO	Clasificación temática del libro

4.8 Tabla: Ejemplar

Campo	Tipo	PK	FK / Restricción	Nulo	Descripción
id_ejemplar	BIGINT IDENTITY	PK		NO	Identificador autogenerado del ejemplar físico
ISBN	VARCHAR(20)		FK→Libro (RESTRICT)	NO	Libro al que pertenece este ejemplar
estado	VARCHAR(30)		CHECK in ('Disponible','Reservado')	NO	Disponibilidad actual del ejemplar; por defecto 'Disponible'

4.9 Tabla: Libro_autor

Campo	Tipo	PK	FK / Restricción	Nulo	Descripción
id_autor	BIGINT	PK, FK→Autor (CASCADE)		NO	Autor relacionado con el libro
ISBN	VARCHAR(20)	PK, FK→Libro (CASCADE)		NO	Libro relacionado con el autor

4.10 Tabla: Prestamo

Campo	Tipo	PK	FK / Restricción	Nulo	Descripción
id_prestamo	BIGINT IDENTITY	PK		NO	Identificador autogenerado del préstamo
id_ejemplar	BIGINT		FK→Ejemplar (RESTRICT)	NO	Ejemplar físico prestado
id_socio	BIGINT		FK→Socio (RESTRICT)	NO	Socio que recibe el préstamo
id_empleado_prestamo	BIGINT		FK→Empleado (RESTRICT)	NO	Empleado que entrega el ejemplar
fecha_prestamo	DATE		CHECK fecha_limite >= fecha_prestamo	NO	Fecha en que se realiza el préstamo
fecha_limite	DATE			NO	Fecha máxima permitida para la devolución
fecha_devolucion	DATE		CHECK >= fecha_prestamo	SI	Fecha real de devolución; NULL si aún no se devuelve
id_empleado_devolucion	BIGINT		FK→Empleado (RESTRICT)	SI	Empleado que recibe la devolución; NULL hasta que ocurra
estado_prestamo	VARCHAR(20)		CHECK in ('Prestado','Devuelto','Vencido')	NO	Estado actual del préstamo; por defecto 'Prestado'

4.11 Tabla: Multa

Campo	Tipo de Dato	PK	FK / Restricción	Nulo	Descripción
id_multa	BIGINT IDENTITY	PK		NO	Identificador autogenerado de la multa
id_prestamo	BIGINT		FK→Prestamo (RESTRICT/CASCADE), UNIQUE	NO	Préstamo que originó la multa; máximo una multa por préstamo
id_tarifa	BIGINT		FK→Tarifa_multa (RESTRICT/CASCADE)	NO	Tarifa aplicada para calcular el monto
dias_retraso	INT		CHECK > 0	NO	Días de retraso respecto a la fecha límite
monto_total	NUMERIC(8,2)		CHECK >= 0	NO	Monto total calculado de la multa
estado_pago	VARCHAR(20)		CHECK in ('Pendiente','Pagada')	NO	Estado del pago de la multa; por defecto 'Pendiente'

5. Consultas SQL Frecuentes

5.1 Top 20 libros más prestados los últimos 3 meses

Muestra los 20 libros que han sido prestados con mayor frecuencia durante los últimos 90 días, ordenados de mayor a menor cantidad de préstamos.

```
select l.titulo, count(p.id_ejemplar ) veces_prestado
from prestamo p
join ejemplar e on p.id_ejemplar = e.id_ejemplar
join libro l on e.isbn = l.isbn
where current_date - p.fecha_prestamo <= 90
group by l.titulo
order by veces_prestado desc
limit 20;
```

5.2 Socios con multas pendientes

Lista los socios que tienen multas sin pagar, mostrando su nombre, los días de retraso y el monto total pendiente.

```
select s.nombre || ' ' || s.apellido nombre_socio , m.dias_retraso, m.monto_total
from multa m
join prestamo p on m.id_prestamo = p.id_prestamo
join socio s on p.id_socio = s.id_socio
where m.estado_pago = 'Pendiente'
order by dias_retraso;
```

5.3 Autores con mayor número de títulos en catálogo

Presenta los 10 autores que tienen más libros registrados en el catálogo de la biblioteca.

```
select a.id_autor, a.nombre, count(*) numero_titulos
from libro l
join libro_autor la on l.isbn = la.isbn
join autor a on la.id_autor = a.id_autor
group by a.nombre, a.id_autor
order by numero_titulos desc
limit 10;
```

5.4 Ejemplares que nunca han sido prestados

Identifica los ejemplares disponibles en la biblioteca que no han sido prestados ninguna vez.

```
select e.id_ejemplar, l.titulo
from ejemplar e
join libro l on e.isbn = l.isbn
left join prestamo p on e.id_ejemplar = p.id_ejemplar
where p.id_ejemplar is null;
```

5.5 Empleado que ha procesado más préstamos en el mes

Muestra el empleado que ha registrado la mayor cantidad de préstamos durante el mes actual.

```
select e.id_empleado , e.nombre , count(p.id_prestamo ) prestamos
from empleado e
join prestamo p on e.id_empleado = p.id_empleado_prestamo
where extract(month from p.fecha_prestamo ) = extract(month from current_date)
and extract(year from p.fecha_prestamo ) = extract(year from current_date)
group by e.id_empleado, e.nombre
order by prestamos desc
limit 1;
```

5.6 Socios con más libros prestados

Presenta los 10 socios que actualmente tienen la mayor cantidad de libros prestados, ordenados de mayor a menor.

```
select s.id_socio , s.nombre || ' ' || s.apellido nombre_socio, count (p.id_prestamo )
libros_prestados
from socio s
join prestamo p on s.id_socio = p.id_socio
where p.estado_prestamo = 'Prestado'
group by s.id_socio
order by libros_prestados desc
limit 10;
```

5.7 Categoría con más ejemplares

Muestra las tres categorías de libros que cuentan con la mayor cantidad de ejemplares disponibles en la biblioteca.

```
select c.nombre_categoria, count(e.id_ejemplar ) total_ejemplares
from libro l
join categoria c on l.id_categoria = c.id_categoria
join ejemplar e on l.isbn = e.isbn
group by c.nombre_categoria
order by total_ejemplares desc
limit 3;
```

6. Funciones, Procedimientos Almacenados y Disparadores

Las siguientes rutinas en PL/pgSQL automatizan el ciclo de préstamo, devolución y multa de la base de datos GestionBiblioteca, implementadas directamente sobre el motor PostgreSQL.

6.1 Script de Mantenimiento de Secuencias

Antes de insertar datos manualmente (por ejemplo, en restauraciones o cargas masivas), se ejecuta este bloque para evitar colisión de llaves primarias, sincronizando cada secuencia identity con el valor máximo presente en su tabla:

```
select setval(pg_get_serial_sequence('prestamo', 'id_prestamo'), coalesce((select
MAX(id_prestamo) from prestamo), 1));

select setval(pg_get_serial_sequence('ejemplar', 'id_ejemplar'), coalesce((select
MAX(id_ejemplar) from ejemplar), 1));

select setval(pg_get_serial_sequence('socio', 'id_socio'), coalesce((select MAX(id_socio) from
socio), 1));

select setval(pg_get_serial_sequence('multa', 'id_multa'), coalesce((select MAX(id_multa) from
multa), 1));
```

6.2 Funciones

fn_actualizar_limite_prestamo() RETURNS TRIGGER

Función asociada a un trigger BEFORE INSERT sobre Prestamo. Si fecha_prestamo llega vacía, la establece en la fecha actual; en todos los casos calcula fecha_limite sumando 7 días a fecha_prestamo, evitando que el usuario tenga que definir manualmente el plazo de devolución.

```
create or replace function fn_actualizar_limite_prestamo()
```

```

returns trigger as $$
begin
    if new.fecha_prestamo is null then
        new.fecha_prestamo := current_date;
    end if;
    new.fecha_limite := new.fecha_prestamo + 7;
    return new;
end;
$$ language plpgsql;

```

fn_actualiza_estado_prestamo() RETURNS TRIGGER

Función asociada a un trigger AFTER UPDATE sobre Prestamo. Sincroniza el estado del Ejemplar según el nuevo estado_prestamo: si el préstamo pasa a 'Devuelto', el ejemplar vuelve a 'Disponible'; si pasa a 'Vencido', el ejemplar se marca como 'Reservado' (no disponible para nuevos préstamos hasta resolver la mora).

```

create or replace function fn_actualiza_estado_prestamo()
returns trigger
as $$
begin
    if new.estado_prestamo = 'Devuelto' and old.estado_prestamo <> 'Devuelto' then
        update ejemplar
        set estado = 'Disponible'
        where id_ejemplar = new.id_ejemplar;
    end if;
    IF new.estado_prestamo = 'Vencido' and old.estado_prestamo <> 'Vencido' then
        update ejemplar
        set estado = 'Reservado'
        where id_ejemplar = new.id_ejemplar;
    end if;
    return new;
end;
$$ language plpgsql;

```

fn_actualiza_estado_multa() RETURNS TRIGGER

Función asociada a un trigger AFTER UPDATE sobre Multa. Cuando una multa cambia su estado_pago a 'Pagada', actualiza automáticamente el préstamo relacionado: lo marca como 'Devuelto' y registra la fecha_devolucion en la fecha actual, cerrando el ciclo del préstamo.

```

create or replace function fn_actualiza_estado_multa()
returns trigger
as $$
declare
    v_id_empleado_devolucion bigint;
begin
    if new.estado_pago = 'Pagada' and old.estado_pago <> 'Pagada' THEN
        v_id_empleado_devolucion := floor(random() * 15 + 1)::bigint;
        update Prestamo
        set estado_prestamo = 'Devuelto',
            fecha_devolucion = current_date,
            id_empleado_devolucion = v_id_empleado_devolucion
        where id_prestamo = new.id_prestamo;
    end if;
    return new;
end;
$$ language plpgsql;

```

fn_consulta_categoria(p_nombre_categoria VARCHAR) RETURNS TABLE(isbn_libro VARCHAR, titulo_libro VARCHAR)

Función de consulta que retorna el ISBN y título de todos los libros pertenecientes a una categoría dada, uniendo Libro con Categoría por nombre_categoria. Permite consultar el catálogo filtrado por tema sin escribir el JOIN manualmente cada vez.

```

create or replace function fn_consulta_categoria(p_nombre_categoria varchar)
returns table (
isbn_libro varchar,
titulo_libro varchar
)
language plpgsql
as $$
begin
return query
select l.isbn, l.titulo
from libro l
inner join categoria c on l.id_categoria = c.id_categoria
WHERE c.nombre_categoria = p_nombre_categoria;
end;
$$;

```

6.3 Procedimientos Almacenados

sp_calcular_multas(p_id_tarifa BIGINT)

Procedimiento de mantenimiento periódico. Primero marca como 'Vencido' todos los préstamos cuya fecha_limite ya pasó y que aún están en estado 'Prestado'. Luego inserta en Multa una multa por cada préstamo vencido que todavía no tenga una multa registrada, calculando dias_retraso y monto_total con la tarifa indicada por parámetro.

```

create or replace procedure sp_calcular_multas(p_id_tarifa bigint)
language plpgsql
as $$
begin
update prestamo
set estado_prestamo = 'Vencido'
where fecha_limite < current_date and estado_prestamo = 'Prestado';
insert into multa(id_prestamo, id_tarifa, dias_retraso, monto_total)
select
p.id_prestamo,
p.id_tarifa,
(current_date - p.fecha_limite) as dias_retraso, ((current_date - p.fecha_limite) *
t.precio_por_dia) as monto_total
from prestamo p
join tarifa_multa t on t.id_tarifa = p_id_tarifa
where p.estado_prestamo = 'Vencido'
and p.fecha_limite < current_date
and not exists(select 1 from multa m where m.id_prestamo = p.id_prestamo);
end;
$$;

```

sp_reserva_libro(p_id_ejemplar BIGINT, p_id_socio BIGINT, p_id_empleado BIGINT)

Procedimiento que registra un préstamo validando primero la disponibilidad del ejemplar. Si el ejemplar está 'Disponible', inserta el préstamo (con fecha_limite a 7 días) y actualiza el ejemplar a 'Reservado'. Si no está disponible, lanza una excepción indicando el estado actual del ejemplar.

```

create or replace procedure sp_reserva_libro(
p_id_ejemplar bigint,
p_id_socio bigint,
p_id_empleado bigint
)
language plpgsql
as $$
declare
v_estado_ejemplar varchar;
begin
select estado into v_estado_ejemplar
from ejemplar
where id_ejemplar = p_id_ejemplar;
if v_estado_ejemplar = 'Disponible' then
insert into prestamo(id_ejemplar, id_socio, id_empleado, prestamo,
fecha_limite, estado_prestamo)
values(p_id_ejemplar, p_id_socio, p_id_empleado,
current_date, current_date + 7, 'Prestado');

```

```

        update ejemplar
        set estado = 'Reservado'
        where id_ejemplar = p_id_ejemplar;
        raise notice 'Libro reservado con éxito';
    else
        raise exception 'EL ejemplar % no esta disponible (estado actual: %)',
            p_id_ejemplar, v_estado_ejemplar;
    end if;
end;
$$;

```

6.4 Disparadores (Triggers)

trg_actualizar_limite_prestamo — BEFORE INSERT ON Prestamo

Ejecuta `fn_actualizar_limite_prestamo()` antes de insertar cada préstamo, garantizando que todo registro nuevo tenga `fecha_prestamo` y `fecha_limite` calculadas automáticamente (límite = fecha de préstamo + 7 días).

```

create trigger trg_actualizar_limite_prestamo
before insert on prestamo
for each row
execute function fn_actualizar_limite_prestamo();

```

trg_actualiza_estado_prestamo — AFTER UPDATE ON Prestamo

Ejecuta `fn_actualiza_estado_prestamo()` después de cada actualización de Prestamo, manteniendo sincronizado el campo estado de Ejemplar con los cambios de estado_prestamo ('Devuelto' → 'Disponible', 'Vencido' → 'Reservado').

```

create trigger trg_actualiza_estado_prestamo
after update on prestamo
for each row
execute function fn_actualiza_estado_prestamo();

```

trg_actualiza_estado_multa — AFTER UPDATE ON Multa

Ejecuta `fn_actualiza_estado_multa()` después de cada actualización de Multa, cerrando automáticamente el préstamo asociado (estado 'Devuelto' y fecha_devolucion actual) en el momento en que la multa se marca como 'Pagada'.

```

create trigger trg_actualiza_estado_multa
after update on multa
for each row
execute function fn_actualiza_estado_multa();

```